1. RPC

```python
import csv

def processData(value):
    return int(value) * 2

def calculate(value):
    return int(value) + 10

with open('rpc_calls_1.csv','r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        method = row["method_name"]
        value = row["param_value"]

        print("\n[Client] Sending Request")
        print("[MARSHAL] Sending Parameter: ", value)
        print("[DISPATCH] Client Invoking: ", method)

        if method == 'processData':
            result = processData(value)

        elif method == 'calculate':
            result = calculate(value)

        else:
            print("[Error] Method not found: ", method)
            continue

        print("[Server Execution] Processing Parameter")
        print("[Return] Result: ",result)

print("\nRPC Simulation Completed")
```

2. Bully Algorithm

```
import csv
nodes = []

with open('bully_cluster_2.csv','r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        nodes.append([int(row["node_id"]), row["is_alive"],
row["is_current_coordinator"]])

print("Nodes in System: ")
for node in nodes:
    print("Node: ",node[0]," is_alive: ",node[1])

active = [n[0] for n in nodes if n[1] == "TRUE"]

coordinator = max([n[0] for n in nodes if n[2] == "TRUE"], default=0)

if len(active) == 0:
    print("No Active Nodes!")
else:
    print("\nElection Started by Node ",coordinator)

    for n in active:
        if n > coordinator:
            print("Election sent to node ", n)
            print("Node ", n," replies: OK")

    leader = max(active)
    print("\nNew Coordinator: ",leader)
    print("Coordinator Message Broadcasted!")
```

3. Berkeley Clock Synchronization

```
import csv
nodes = []

with open('node_clocks_3.csv','r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        nodes.append([row["node_id"], int(row["clock_time_sec"]),
row["is_faulty"]])

print("\nOriginal Clock Time: ")
for n in nodes:
    print("Node ",n[0],":",n[1])

active = [n for n in nodes if n[2] == "FALSE"]

if len(active) == 0:
    print("No Active Nodes!")
else:
    avg = sum(n[1] for n in active) / len(active)
    print("Average Clock Time: ",avg)

    print("\nClock Adjustments: ")
    for n in active:
        print("Node ",n[0]," Adjustment:",avg-n[1]," seconds")
        n[1] = avg

    print("\nSynchronized Clock Times: ")
    for n in nodes:
        if n[2] == "FALSE":
            print("Node",n[0]," : ",n[1])
        else:
            print("Node",n[0],": Faulty not synchronized")
```

4. Multithreading

```
import csv
import threading
import time

tasks = []
completed = []

lock = threading.Lock()

with open('tasks_4.csv','r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        tasks.append(row)

def process(task):
    name = task["thread_name"]
    task_id = task["task_id"]

    print(name," Started processing", task_id)
    time.sleep(int(task["burst_ms"])/1000)

    with lock:
        print(name," acquired lock")
        task["status"] = "Completed"
        completed.append(task_id)
        print(name," release lock")

threads = []

for task in tasks:
    t = threading.Thread(target=process, args=(task,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("\nCompleted Tasks: ")
for task in completed:
    print(task," - completed")

print("All Task Completed")
```

5. Load Balancing

```
import csv

workers = ["worker_1","worker_2","worker_3"]
load = [0,0,0]
jobs = []

with open('workload_5.csv','r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        jobs.append(row)

for i, job in enumerate(jobs):
    w = i % 3
    job["assigned_worker"] = workers[w]
    job["status"] = "Assigned"

    units = int(job["compute_units"])
    load[w] += units

    print("Job",job["job_id"]," assigned to: ",workers[w])

print("\nWorkers Load")
for i in range(3):
    print(workers[i]," : ",load[i]," units")

print("\nFinal Job Status: ")
for job in jobs:
    print(job["job_id"]," - ",job["assigned_worker"]," - ",job["status"])

print("\nLoad Balancing Completed")
```

6. Vector Clock

```python
import csv

clock = [[0, 0, 0] for i in range(3)]
messages = {}
events = []

# Read CSV file
with open("vector_events_6.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        p = int(row["process_id"][1]) - 1
        action = row["event_action"].upper()
        target = row["target_process"]

        # RECEIVE event
        if action == "RECEIVE":
            sender = int(target[1]) - 1
            received = messages[(sender, p)].pop(0)

            for i in range(3):
                clock[p][i] = max(clock[p][i], received[i])

        # Increment own clock
        clock[p][p] += 1

        # SEND event
        if action == "SEND":
            t = int(target[1]) - 1

            if (p, t) not in messages:
                messages[(p, t)] = []

            messages[(p, t)].append(clock[p].copy())

        # Store event
        events.append((row["event_id"], clock[p].copy()))
        print(row["event_id"], ":", clock[p])


# Compare event relationships
print("\nEvent Relationships:")

for i in range(len(events)):
    for j in range(i + 1, len(events)):
        e1, a = events[i]
        e2, b = events[j]
```

```
        if all(a[k] <= b[k] for k in range(3)) and a != b:
            print(e1, "and", e2, ": Causally Precedes")

        elif all(a[k] >= b[k] for k in range(3)) and a != b:
            print(e1, "and", e2, ": Causally Follows")

        else:
            print(e1, "and", e2, ": Concurrent")

print("\nVector Clock Algorithm Completed!")
```

7. Lamport Logical Clock

```
import csv

clock = {"P1": 0, "P2": 0, "P3": 0}
events = []

with open('events_7.csv.','r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        p = row["process_id"]
        action = row["event_type"]
        received = int(row["initial_clock"])

        if action == "RECEIVE":
            clock[p] = max(clock[p], received)

        clock[p] += 1
        events.append([row["event_id"], p, clock[p]])
        print(row["event_id"], " Clock: ",clock[p])

events.sort(key = lambda x: (x[2], x[1]))
print("\nFinal Causual Order: ")

for event in events:
    print(event[0], " -> ", event[2])

print("\nLamport Clock Completed!")
```

8. Cristian Clock Synchronization

```
import csv

with open('cristian_clocks_8.csv','r') as file:
    reader = csv.DictReader(file)

    for row in reader:
        client = row["client_id"]
        t0 = int(row["t0_send_ms"])
        ts = int(row["t_server_ms"])
        t1 = int(row["t1_recv_ms"])

        rtt = t1 - t0
        estimated = ts + (rtt/2)
        adjustment = estimated - t1

        print("\nClient: ",client)
        print("RTT: ",rtt, "ms")
        print("Estimated Server Time: ",estimated,"ms")
        print("Clock Adjustment: ",adjustment,"ms")

print("Cristian Algorithm Completed")
```

9. Ring Algorithm

```
import csv

nodes = []
with open('ring_nodes_15.csv','r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        if row["is_active"].upper() == "TRUE":
            nodes.append(int(row["node_id"]))

nodes.sort()
print("Coordinator Failed!")
print("\nElection token Circulating")

for node in nodes:
    print("Token passed to node ", node)

if nodes:
    leader = max(nodes)
    print("\nNew Coordinator: ",leader)
    print("Coordinator message broadcasted!")
else:
    print("No Active nodes available")
```

10. Scatter-Gather

```
import csv
import threading

A = []
result = []

with open('matrix_22.csv','r',) as file:
    reader = csv.DictReader(file)
    for row in reader:
        A.append([
            int(row["col_0"]),
            int(row["col_1"]),
            int(row["col_2"]),
            int(row["col_3"]),
        ])

B = A
n = len(A)
result = [[0] * n for i in range(n)]

def multiply(i):
    print("Worker ",i," computing row", i)

    for j in range(n):
        for k in range(n):
            result[i][j] += A[i][k] * B[j][k]

threads = []
print("\nMaster Scattering Rows")

for i in range(n):
    t = threading.Thread(target=multiply, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("\nMaster gathering results")
print("Final Matrix:")

for row in result:
    print(row)

print("\nScatter-Gather Completed!")
```